Second Year Engineering

Database Management System

(23CSE2022)

Class - S.Y.

AY 2026-2027

SEM-I

Database Management System		
Teaching Scheme	**Credit Scheme**	**Examination Scheme and Marks**
Lecture: 03 Hours/Week	03	Continues Assessment (TH): 50 Marks End_Semester_Examination (TH): 50 Marks

Unit – I

Introduction to Database Management Systems





Syllabus

- Database system, purpose of database system,
- view of data, relational databases, database architecture,
- Database Concepts,
- Three-schema architecture of a database, Data Models,
- ER model, Relational Model, ER to Table Conversion

Introduction :

Database

- A collection of related data that is stored in a central location or in multiple locations
- A database is a structured collection of data.

Data hierarchy: Structure and organization of data involving fields, records, and files

Database Management System (DBMS)

- Software for creating, storing, maintaining, and accessing database files
- Makes using databases more efficient

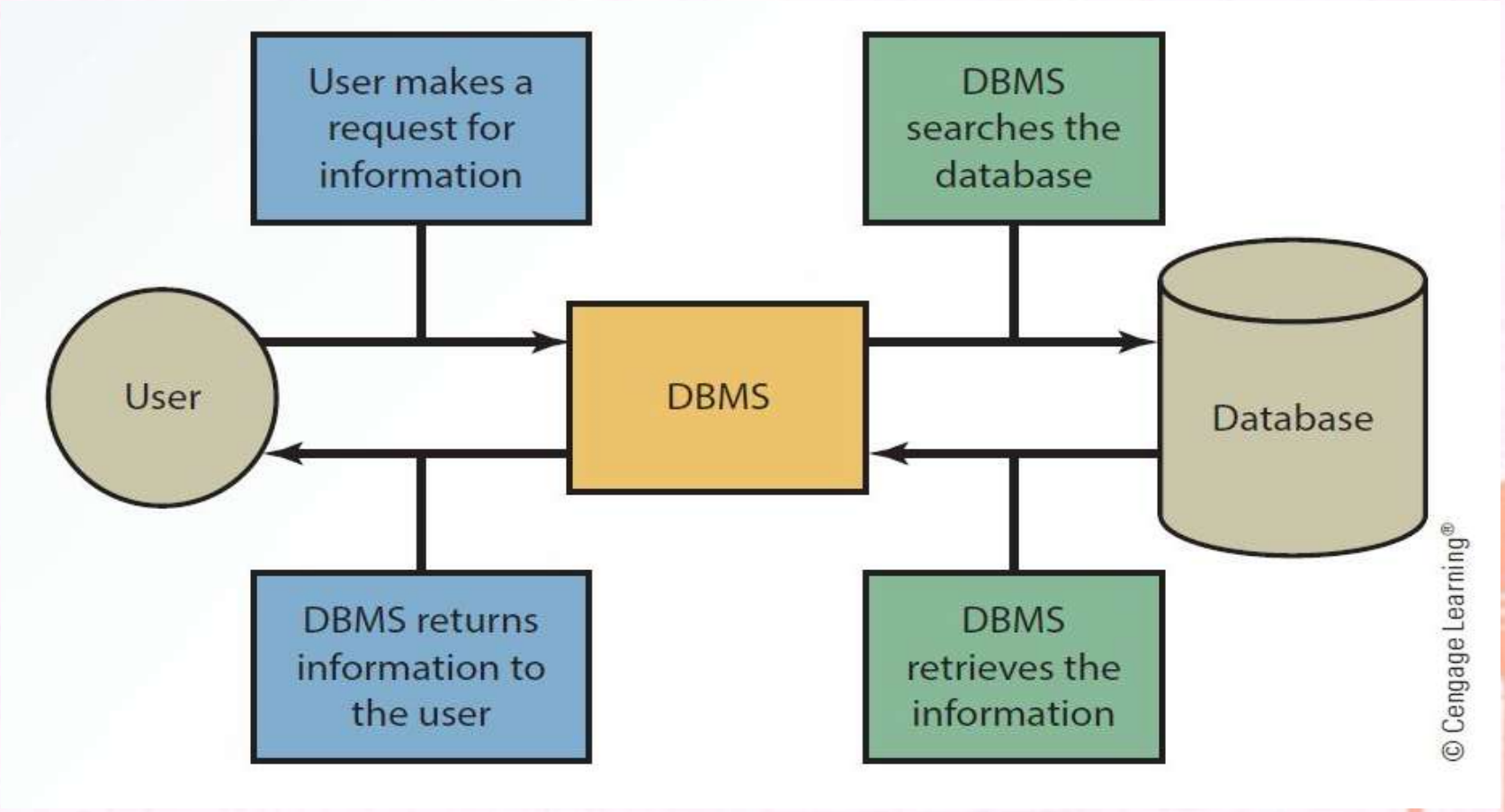
Database Applications Examples

- Enterprise Information
  - Sales: customers, products, purchases
  - Accounting: payments, receipts, assets
  - Human Resources: Information about employees, salaries, payroll taxes.
- Manufacturing: management of production, inventory, orders, supply chain.
- Banking and finance
  - customer information, accounts, loans, and banking transactions.
  - Credit card transactions

Database Applications Examples

- Universities: registration, grades
- Airlines: reservations, schedules
- Telecommunication: records of calls, texts, and data usage, generating monthly bills, maintaining balances on prepaid calling cards
- Web-based services
  - Online retailers: order tracking, customized recommendations
  - Online advertisements

Interaction Between the User, DBMS and Database



Interaction Between the User, DBMS and Database

- Working of a DBMS (Database Management System)
- User makes a request
  The user asks for information by entering a query or request.
- Request goes to the DBMS
  The DBMS receives the user's request and processes it.
- DBMS searches the database
  The DBMS checks the database to find the required data.
- DBMS retrieves the information
  After finding the matching data, the DBMS collects the required information from the database.
- DBMS returns the information to the user
  Finally, the DBMS sends the retrieved information back to the user in an easy-to-understand format.

Purpose of Database Systems/ Disadvantage of File-oriented system

Data Redundancy:

- It is possible that the same information may be duplicated in different files. this leads to data redundancy results in memory wastage.

Data Inconsistency:

- Because of data redundancy, it is possible that data may not be in consistent state.

Limited Data Sharing:

- Data are scattered in various files. Also different files may have different formats and these files may be stored in different folders may be of different departments.
- So, due to this data isolation, it is difficult to share data among different applications.

Integrity Problems:

- Data integrity means that the data contained in the database in both correct and consistent. for this purpose the data stored in database must satisfy correct and constraints.
  - Eg. If a user enters a mobile number in the incorrect format.
  - If the user attempts to remove a record from a database, however, that record is referenced in another table as a relationship

Difficulty in Accessing Data:

Accessing data is not convenient and efficient in file processing system.

Atomicity Problems:

- Any operation on database must be atomic.
- this means, it must happen in its entirely or not at all.

Concurrent Access Anomalies:

- Multiple users are allowed to access data simultaneously. this is for the sake of better performance and faster response.

Security Problems:

- Database should be accessible to users in limited way.
- Each user should be allowed to access data concerning his requirements only

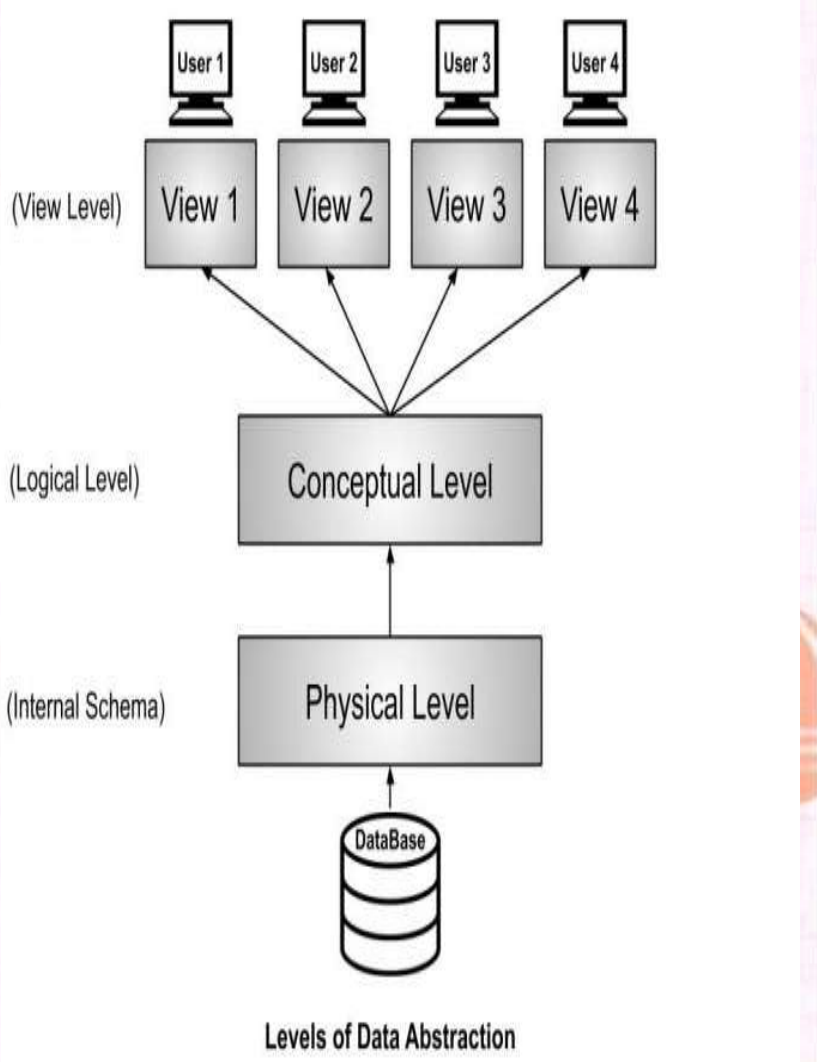
View of Data

- A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data.
- A major purpose of a database system is to provide users with an abstract view of the data. That is, the system hides certain details of how the data are stored and maintained.
  - **Data Abstraction**
  - **Instances and Schemas**
  - **Data Models**

Data Abstraction

There are mainly three levels of data abstraction:

- **Internal Level**: Actual PHYSICAL storage structure and access paths.
- **Conceptual or Logical Level**: Structure and constraints for the entire database
- **External or View level**: Describes various user views



Levels of Data Abstraction



- **Example**: Let's say we are storing customer information in a customer table. At physical level these records can be described as blocks of storage (bytes, gigabytes, terabytes etc.) in memory. These details are often hidden from the programmers.
- Gmail account

Physical level: describes how a record (e.g., instructor) is stored.

Logical level: describes data stored in database, and the relationships among the data.

```
type instructor = record
        ID : string;
    name : string;
    dept_name : string;
    salary : integer;
        end;
```

View level: application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

Instance:

- The collection of information stored in the database at a particular moment is called an instance of the database.

Schemas: The Overall Structure of a database is called Schemas

Logical Schema : *logical structure of DB*
Example- Customers Acc

Physical Schema: *Physical structure of DB*
Example- Files

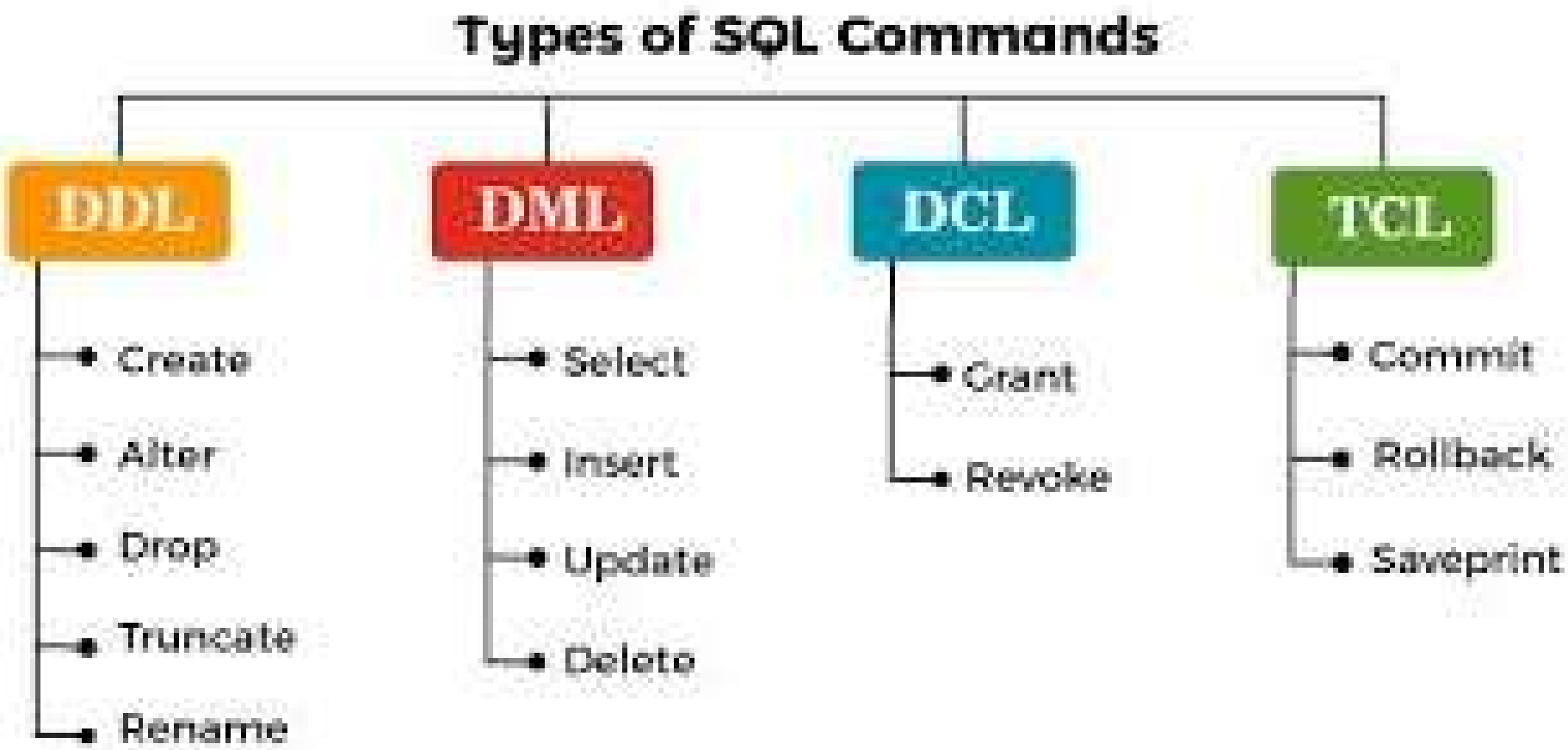
Data Independence

Logical Data Independence :Ability to modify the logical schema without affecting the external schema/application program

Physical Data Independence

Ability to modify the physical schema without changing the logical schema.

- Applications depend on the logical schema
- In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.


Types of SQL Commands
DDL
Create
Alter
Drop
Truncate
Rename
DML
Select
Insert
Update
Delete
DCL
Grant
Revoke
TCL
Commit
Rollback
Saveprint

Relational Databases

- A relational database is based on the relational model and uses a collection of tables to represent both data and the relationships among those data.
  - Tables
  - Data-Manipulation Language
  - Data-Definition Language

Data-Definition Language

- Specification notation for defining the database schema
- Example: create table instructor (

  ID char(5),

  name varchar(20),

  dept_name varchar(20),

  salary numeric(8,2))
- DDL compiler generates a set of table templates stored in a data dictionary
- Data dictionary contains metadata (i.e., data about data)

DDL Command

- CREATE:

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype,
    ....
);
```

- DROP:

```
DROP TABLE table_name;
```

- ALTER:

```
ALTER TABLE table_name
ADD column_name datatype;
```

- TRUNCATE:

```
TRUNCATE TABLE table_name;
```

Create

- Step 1:

```
>SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| CollegeDB          |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
```

- Step 2:

```
-- Create a database
>CREATE DATABASE CollegeDB;
-- Select the database
>USE CollegeDB;
-- Create a table
>CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    Age INT,
    Department VARCHAR(30)
);
```

```
-- Display all databases
 >SHOW DATABASES;

-- Display all tables
 >SHOW TABLES;

 --Display the Table
 >DESC Student;
```

Output:

```
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| StudentID   | int         | NO   | PRI | NULL    |       |
| StudentName | varchar(50) | YES  |     | NULL    |       |
| Age         | int         | YES  |     | NULL    |       |
| Department  | varchar(30) | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
```

Alter:

Add a Phone column to the Student table.

```
>ALTER TABLE Student
ADD PhoneNumber VARCHAR(15);
>DESC Student;
```

```
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| StudentID   | int         | NO   | PRI | NULL    |       |
| StudentName | varchar(50) | YES  |     | NULL    |       |
| Age         | int         | YES  |     | NULL    |       |
| Department  | varchar(30) | YES  |     | NULL    |       |
| PhoneNumber | varchar(15) | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
```

- Mysql>ALTER TABLE table_name
-> DROP COLUMN column_name_1,
-> DROP COLUMN column_name_2,
-> ;

- mysql> alter table students drop column phonenumber;
- mysql> desc students;

```
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| st_roll_no | int         | YES  |     | NULL    |       |
| st_name    | varchar(10) | YES  |     | NULL    |       |
| st_address | varchar(10) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
3 rows in set (0.00 sec)
```

- mysql> alter table students rename to student_info;
- mysql> desc students;

ERROR 1146 (42S02): Table 'ddl.students' doesn't exist

- mysql> desc students_info;

```
+-----------+-------------+------+-----+---------+-------+
| Field     | Type        | Null | Key | Default | Extra |
+-----------+-------------+------+-----+---------+-------+
| st_roll_no | int        | YES  |     | NULL    |       |
| st_name    | varchar(10) | YES  |     | NULL    |       |
| st_address | varchar(10) | YES  |     | NULL    |       |
+-----------+-------------+------+-----+---------+-------+
```

- mysql> alter table student_info rename column st_address to st_pincode;
- mysql> desc student_info;

```
+-----------+------------+------+-----+---------+-------+
| Field     | Type       | Null | Key | Default | Extra |
+-----------+------------+------+-----+---------+-------+
| st_roll_no | int        | YES  |     | NULL    |       |
| st_name    | varchar(10) | YES  |     | NULL    |       |
| st_pincode | varchar(10) | YES  |     | NULL    |       |
```



Truncate:

The TRUNCATE command deletes **all records** from the table but **keeps the table structure**

- mysql> truncate table student_info;
- mysql> desc student_info;

```
+-----------+------------+------+-----+---------+-------+
| Field     | Type       | Null | Key | Default | Extra |
+-----------+------------+------+-----+---------+-------+
| st_roll_no | int        | YES  |     | NULL    |       |
| st_name    | varchar(10) | YES  |     | NULL    |       |
| st_pincode | varchar(10) | YES  |     | NULL    |       |
+-----------+------------+------+-----+---------+-------+
```

Drop:

- mysql> drop table student_info;

Query OK, 0 rows affected (0.02 sec)

- mysql> desc student_info;

ERROR 1146 (42S02): Table 'ddl.student_info' doesn't exist

Data-Manipulation Language

- There are basically two types of data-manipulation language

  Procedural DML -- require a user to specify what data are needed and how to get those data.

  - **Declarative DML** -- require a user to specify what data are needed without specifying how to get those data.

- Declarative DMLs are usually easier to learn and use than are procedural DMLs.
- Declarative DMLs are also referred to as non-procedural DMLs
- The portion of a DML that involves information retrieval is called a query language.



Procedural DML

A language in which the user specifies both what data is required and how to retrieve it. The programmer writes the steps for data retrieval.

- Example: PL/SQL, Embedded SQL

```
Open Student table
Read each record
If Marks > 80
   Display Name
End
```



Non-Procedural DML:

- A language in which the user specifies **only what data is required**, while the DBMS determines how to retrieve it.
- Example:SQL(Select, Update,Insert, Delete)

```
SELECT Name
FROM Student
WHERE Marks > 80;
```

DML

- DML is short name of **Data Manipulation Language** which deals with data manipulation.
- SELECT - retrieve data from a database
- INSERT - insert data into a table
- UPDATE - updates existing data within a table
- DELETE - Delete all records from a database table

DML Command

INSERT:

```
Insert into <table_name> (column list) values (column values);
```

Examples:

```
Insert into Employee(Emp_id, Emp_name) values (001, " bhanu");
Insert into Employee(Emp_id, Emp_name) values (002, " hari");
Insert into Employee(Emp_id, Emp_name) values (003, " bob");
```

SELECT:

```
SELECT * from <table_name>;
```

Example:

```
SELECT * from Employee;
```

DELETE:

```
Delete from <table_name>WHERE condition;
```

Examples:

```
DELETE from Employee WHERE Emp_id=002;
```

UPDATE:

```
UPDATE <table_name> SET column_number =value_number WHERE condition;
```

Examples

```
UPDATE Employee SET Emp_name= Ram WHERE Emp_id= 001;
```

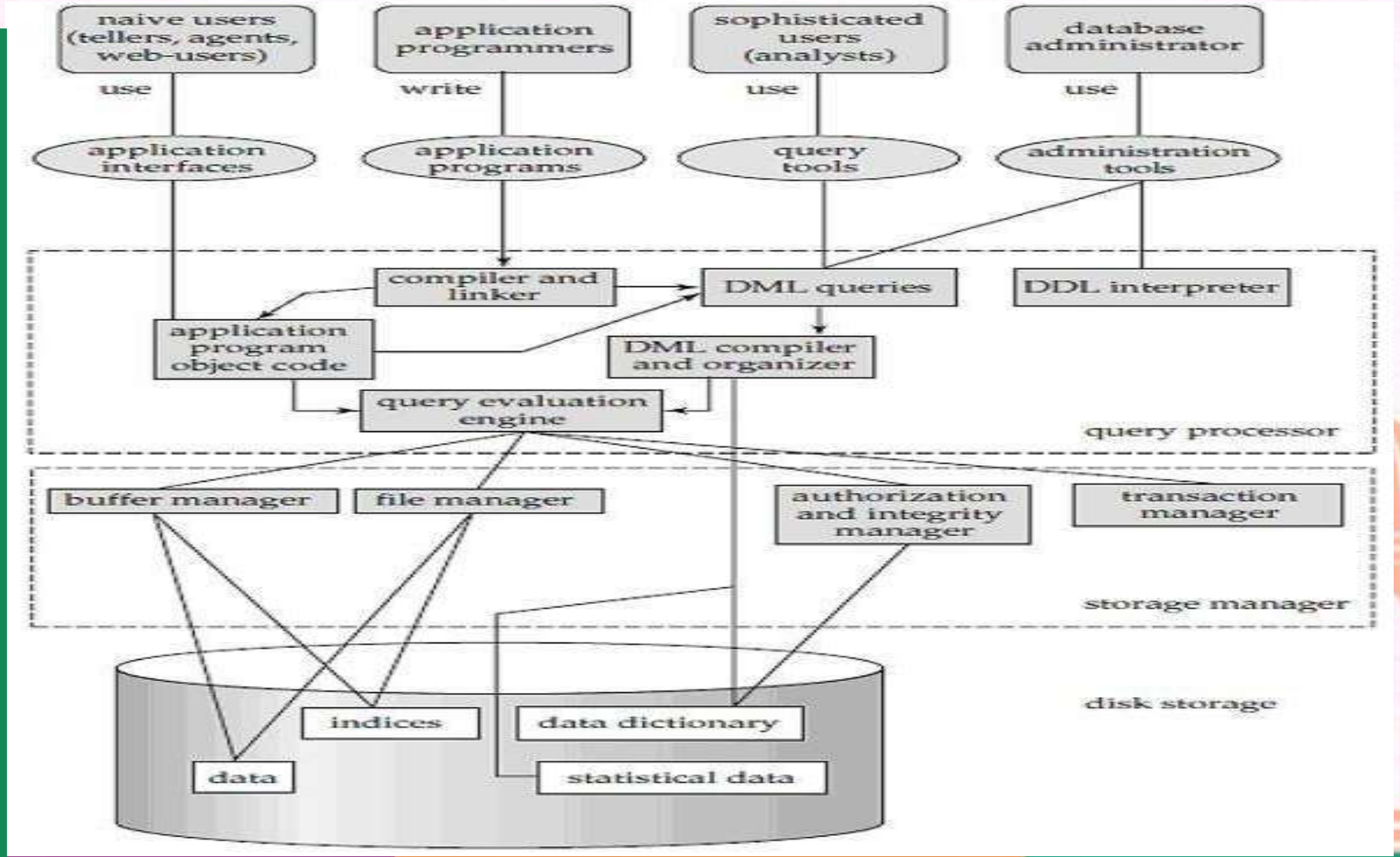
Database Architecture

- Centralized databases
  - One to a few cores, shared memory
- Client-server,
  - One server machine executes work on behalf of multiple client machines.
- Parallel databases
  - Many core shared memory
  - Shared disk
  - Shared nothing
- Distributed databases
- Geographical distribution
- Schema/data heterogeneity

Database Architecture

- The architecture of a database system is greatly influenced by the underlying computer system on which the database system runs.
- Database systems can be centralized, or client-server, where one server machine executes work on behalf of multiple client machines.
- A database system is partitioned into modules that deal with each of the responsibilities of the overall system.
- The functional components of a database system can be divided into
  - The storage manager,
  - The query processor component,
  - The transaction management component

Database Architecture



The Query Processor

- The query processor components include
- • **DDL interpreter**, which interprets DDL statements and records the definitions in the data dictionary.
- • **DML compiler,** which translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.
- A query can usually be translated into any of a number of alternative evaluation plans that all give the same result. The DML compiler also performs query optimization, that is, it picks the lowest cost evaluation plan from among the alternatives.
- • **Query evaluation engine**, which executes low-level instructions generated by the DML compiler.

Storage Manager

- A storage manager is a program module that provides the interface between the low level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible for the interaction with the file manager.
- The raw data are stored on the disk using the file system, which is usually provided by a conventional operating system. The storage manager translates the various DML statements into low-level file-system commands.
- Thus, the storage manager is responsible for storing, retrieving, and updating data in the database.

The storage manager components include:

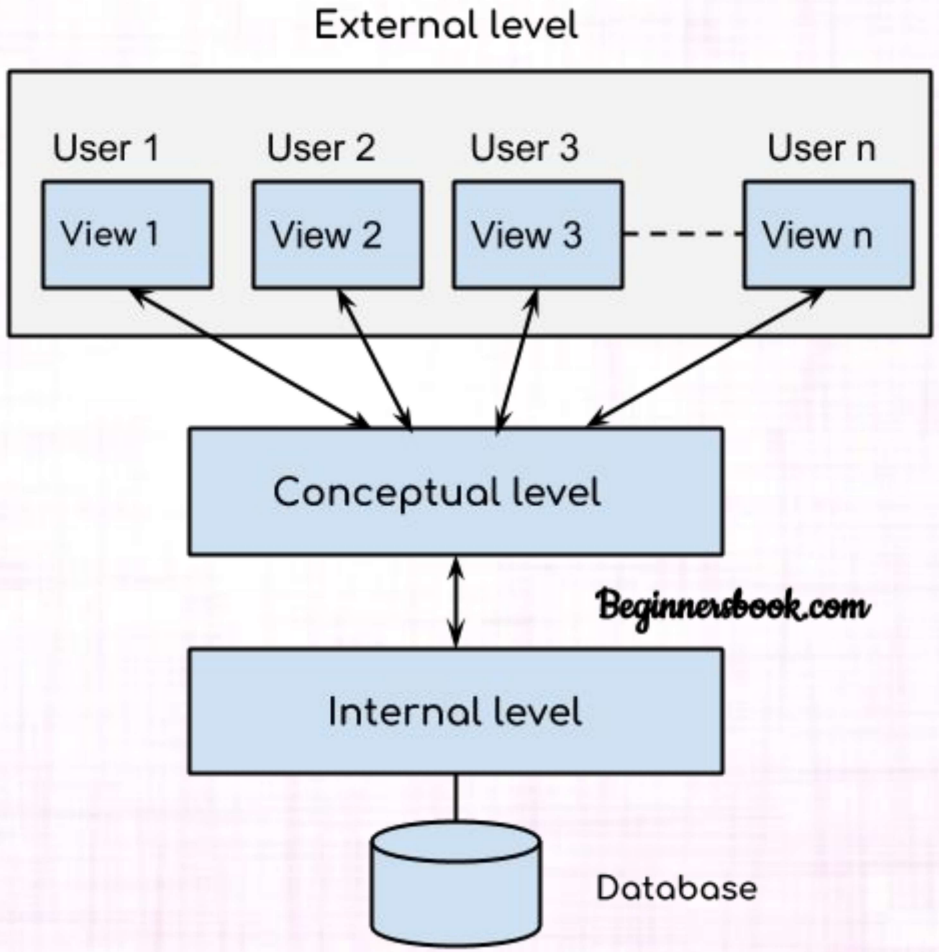
- **Authorization and integrity manager**, which tests for the satisfaction of integrity constraints and checks the authority of users to access data.
- **Transaction manager**, which ensures that the database remains in a consistent (correct) state despite system failures, and that concurrent transaction executions proceed without conflicting.
- **File manager**, which manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- **Buffer manager,** which is responsible for fetching data from disk storage into main memory, and deciding what data to cache in main memory.

- The storage manager implements several data structures as part of the physical system implementation:
- **Data files**, which store the database itself.
- **Data dictionary,** which stores metadata about the structure of the database, in particular the schema of the database.
- **Indices,** which provide fast access to data items that hold particular values

DBMS Three Level Architecture

This architecture has three levels:

1. External level
2. Conceptual level
3. Internal level



1. External level

It is also called view level. The reason this level is called "view" is because several users can view their desired data from this level which is internally fetched from database with the help of conceptual and internal level mapping.

2. Conceptual level

It is also called **logical level**. The whole design of the database such as relationship among data, schema of data etc. are described in this level.

3. Internal level

- This level is also known as physical level. This level describes how the data is actually stored in the storage devices. This level is also responsible for allocating space to the data. This is the lowest level of the architecture.



Data Models

- data model: a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints
- Data Model gives us an idea that how the final system will look like after its complete implementation.
- Data Models are used to show how data is stored, connected, accessed and updated in the database management system.
- Data models define how the logical structure of a database is modeled.
- It is classified into 3 types:

1. Conceptual Data Models -- ER Model
2. Representational Data Models – Relational Model
3. Physical Data Models



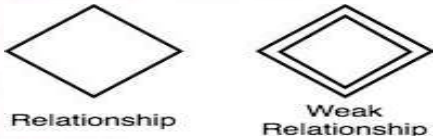
Entity-Relationship Model

- ER model stands for an Entity-Relationship model. It is a high-level data model. This model is used to define the data elements and relationship for a specified system.
- It develops a conceptual design for the database.
- We use the ER diagram as a visual tool to represent an ER Model. ER diagram has the following three components:
- ***Entities:*** Entity is a real-world thing. It can be a person, place, or even a concept. *Example:* Teachers, Students, Course, Building, Department, etc. are some of the entities of a School Management System.
- ***Attributes:*** An entity contains a real-world property called attribute. This is the characteristics of that attribute. *Example:* The entity teacher has the property like teacher id, salary, age, etc.
- ***Relationship:*** Relationship tells how two entities are related. *Example:* Teacher works for a department.

- **Entity-**Simple rectangular box represents an Entity.



- **Relationships between Entities - Weak and Strong**



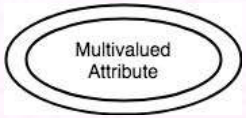
- **Weak Entity-**A weak Entity is represented using double rectangular boxes



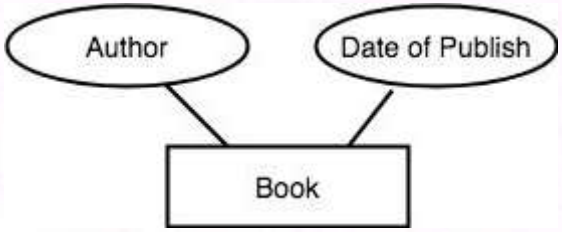
- **Derived Attribute for any Entity**



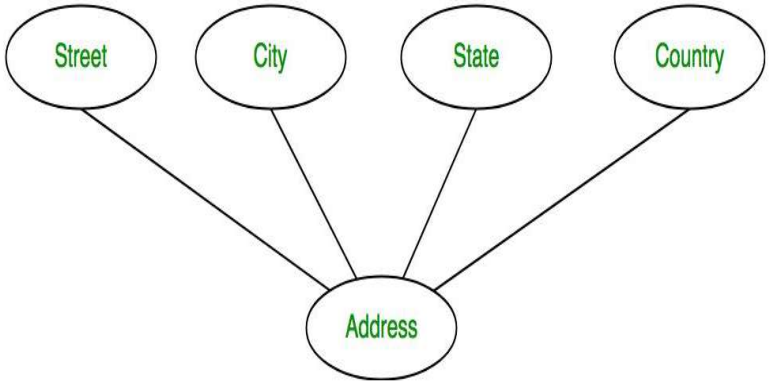
- **Multivalued Attribute for any Entity**

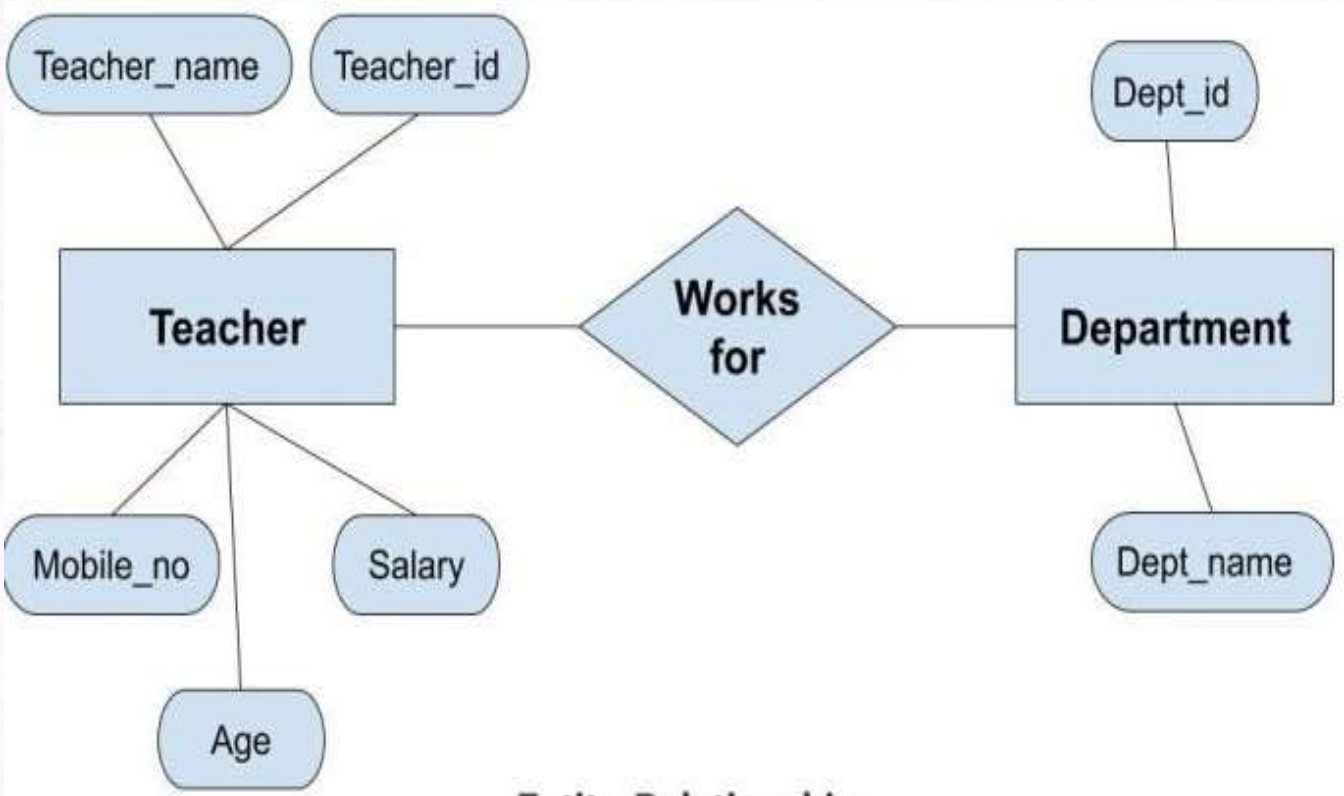


- **Attributes for any Entity**



- **Composite Attribute for any Entity**





Entity-Relationship Model

Advantages of ER Model

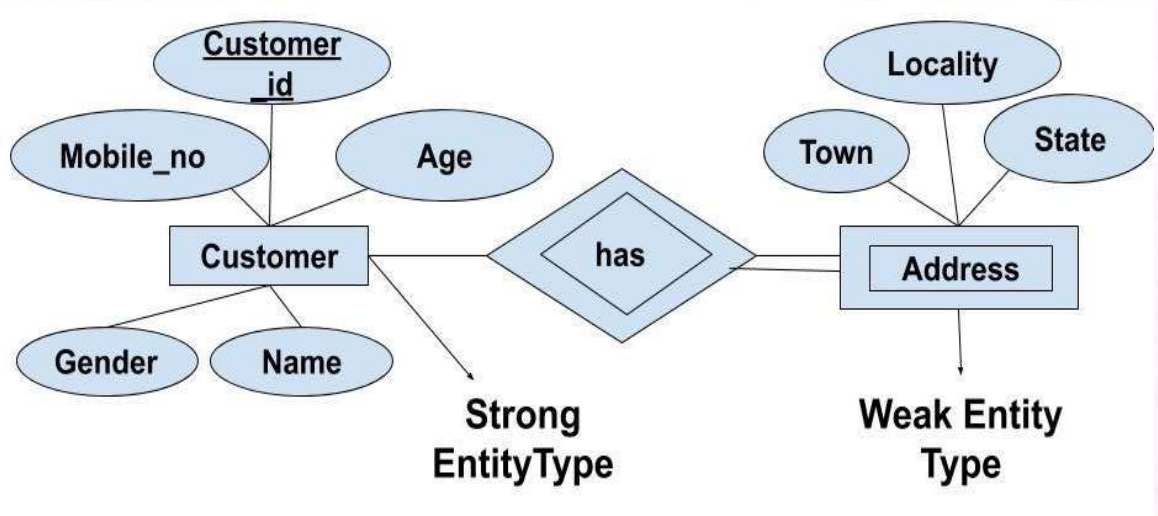
- ***Simple:*** Conceptually ER Model is very easy to build. If we know the relationship between the attributes and the entities we can easily build the ER Diagram for the model.
- ***Effective Communication Tool***: This model is used widely by the database designers for communicating their ideas.
- ***Easy Conversion to any Model***: This model maps well to the relational model and can be easily converted relational model by converting the ER model to the table. This model can also be converted to any other model like network model, hierarchical model etc.

Types of Entity type

- Strong Entity Type
- Weak Entity Type

- ***Strong Entity Type:*** Strong entity are those entity types which has a key attribute. The primary key helps in identifying each entity uniquely
- ***Weak Entity Type***: Weak entity type doesn't have a key attribute. Weak entity type can't be identified on its own. It depends upon some other strong entity for its distinct identity.

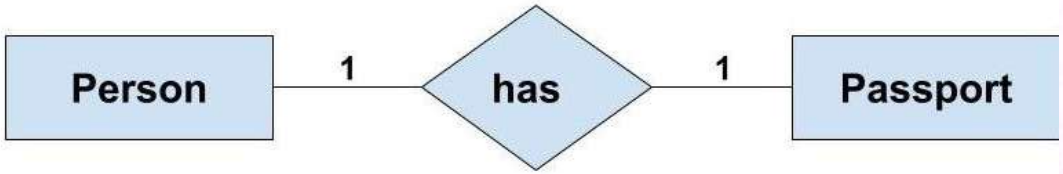
- If we have two tables of Customer and Address.
- Here we cannot identify the address uniquely as there can be many customers from the same locality.
- So, for this, we need an attribute of Strong Entity Type i.e 'Customer' here to uniquely identify entities of 'Address' Entity Type.



Relationship in DBMS

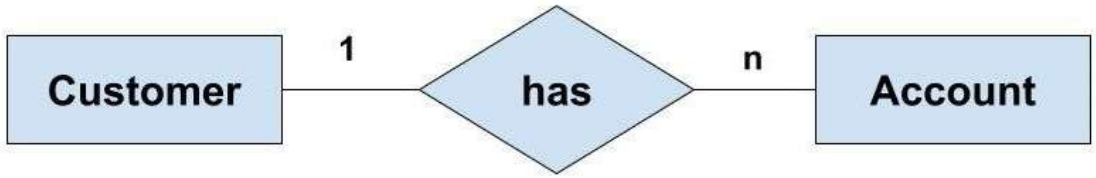
- Any association between two entity types is called a relationship.
- One-to-One Relationship
- One-to-Many or Many-to-One Relationship
- Many-to-Many Relationship
- **One-to-One Relationship**

Such a relationship exists when each record of one table is related to only one record of the other table.



- **One-to-Many or Many-to-One Relationship**

Such a relationship exists when each record of one table can be related to one or more than one record of the other table



If there are two entity type 'Customer' and 'Account' then each 'Customer' can have more than one 'Account' but each 'Account' is held by only one 'Customer'. In this example, we can say that each Customer is associated with many Account. So, it is a one-to-many relationship. But, if we see it the other way i.e many Account is associated with one Customer then we can say that it is a many-to-one relationship.

Many-to-Many Relationship

- If there are two entity type ‘Customer’ and ‘Product’ then each customer can buy more than one product and a product can be bought by many different customers.



Types of Participation Constraints-

- Total participation
- Partial participation
- **1. Total Participation-**
- It specifies that each entity in the entity set must compulsorily participate in at least one relationship instance in that relationship set.
- That is why, it is also called as **mandatory participation.**
- Total participation is represented using a double line between the entity set and relationship set.



- Double line between the entity set "Student" and relationship set "Enrolled in" signifies total participation.
- It specifies that each student must be enrolled in at least one course.

- **2. Partial Participation-**
- It specifies that each entity in the entity set may or may not participate in the relationship instance in that relationship set.
- That is why, it is also called as **optional participation.**
- Partial participation is represented using a single line between the entity set and relationship set.



- Single line between the entity set “Course” and relationship set “Enrolled in” signifies partial participation.
- It specifies that there might exist some courses for which no enrollments are made.

Types of Attributes-

- Simple attributes
- Composite attributes
- Single valued attributes
- Multi valued attributes
- Derived attributes
- Key attributes

Types of Attributes-

- Simple attributes :

  Cannot be divided into smaller parts. It stores a single piece of information.

  Example: Age, Gender, Salary.

- Composite attributes :

  Can be divided into smaller sub-attributes.

  Example: Name → First Name, Middle Name, Last Name. Address → House No., Street, City, State, PIN Code.

- Single valued attributes : Has only one value for each entity.

  Example: Roll Number, Date of Birth, Aadhaar Number.

Types of Attributes-

- Multi valued attributes: Can have multiple values for a single entity.
- Example: Phone Numbers, Email IDs, Skills.
- Derived attributes: Its value is calculated from another attribute.
- It is usually not stored in the database.
- Example: Age is derived from Date of Birth.
- Key attributes: Uniquely identifies each record in an entity. No two records can have the same key value.
- Example: Student ID, Employee ID, Roll Number.

Relational Model

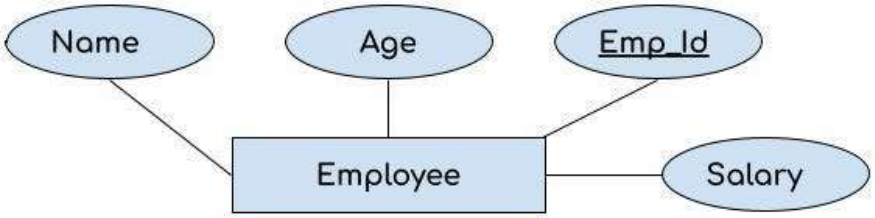
- Relational Model is the most widely used model.
- In this model, the data is maintained in the form of a two-dimensional table.
- All the information is stored in the form of row and columns. The basic structure of a relational model is tables.

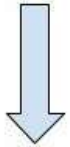
Emp_id	Emp_name	Job_name	Salary	Mobile_no	Dep_id	Project_id
AfterA001	John	Engineer	100000	9111037890	2	99
AfterA002	Adam	Analyst	50000	9587569214	3	100
AfterA003	Kande	Manager	890000	7895212355	2	65

EMPLOYEE TABLE

ER to Table conversion

- Entity type becomes a table.
- All single-valued attribute becomes a column for the table.
- A key attribute of the entity type represented by the primary key.
- Composite attribute represented by components.(student address is a composite attribute, It contains CITY, PIN, DOOR#, STREET, and STATE, In the STUDENT table, these attributes can merge as an individual column.)
- Derived attributes are not considered in the table.





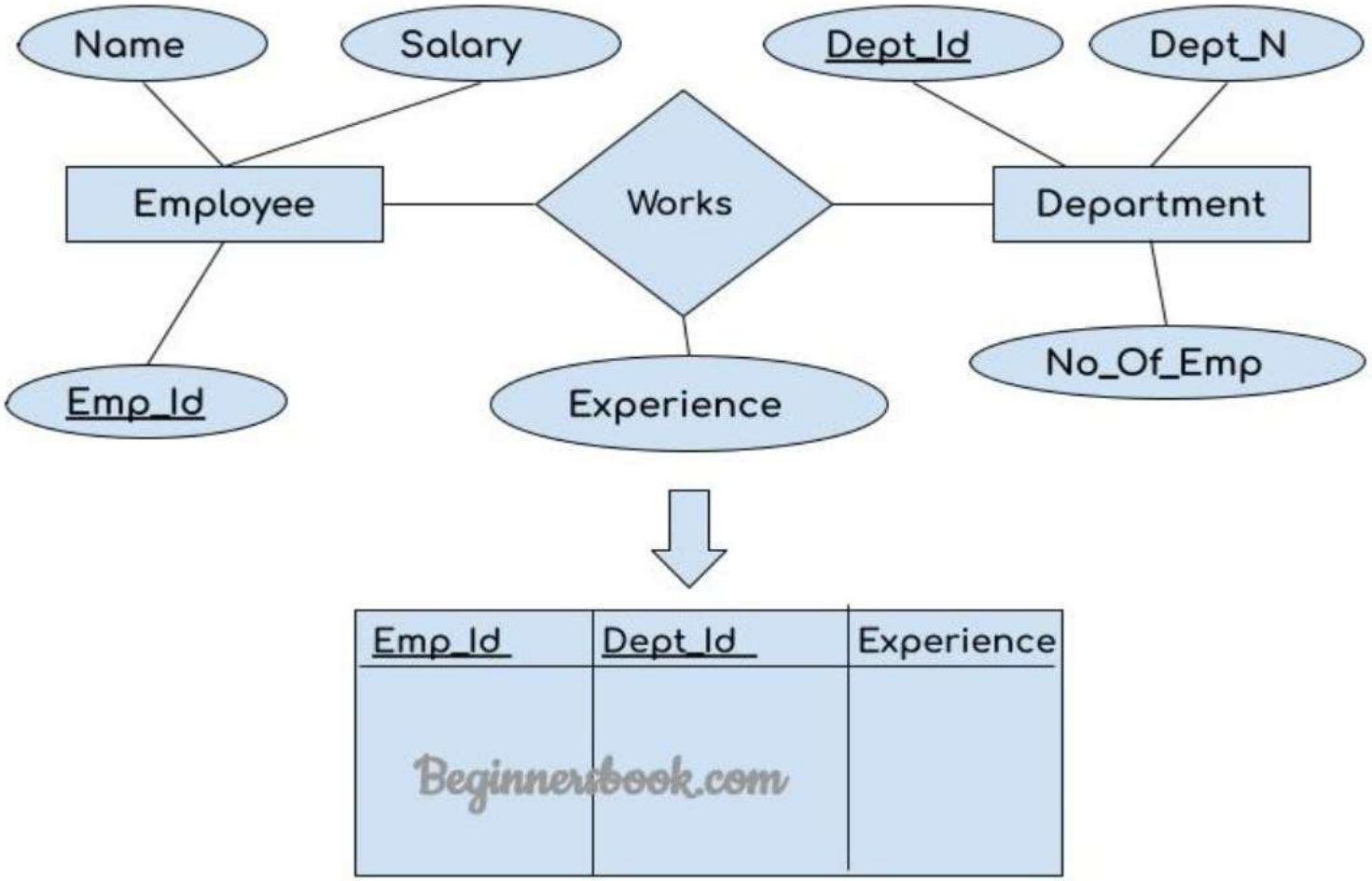
Emp_Id	Name	Age	Salary

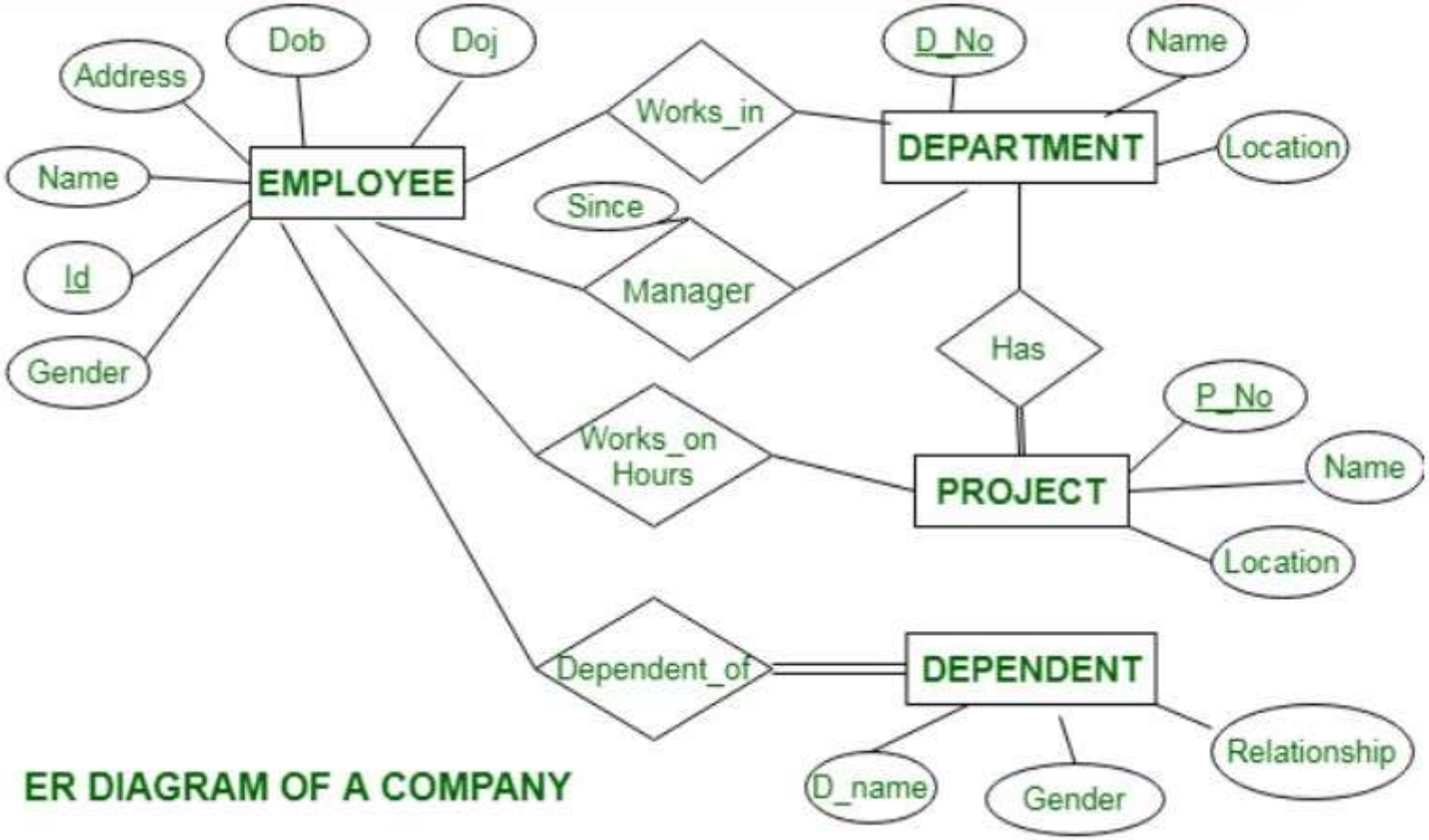
Beginnersbook.com

Table Schema: (Emp_id, Name, Age, Salary)

Relationship Set to Table conversion

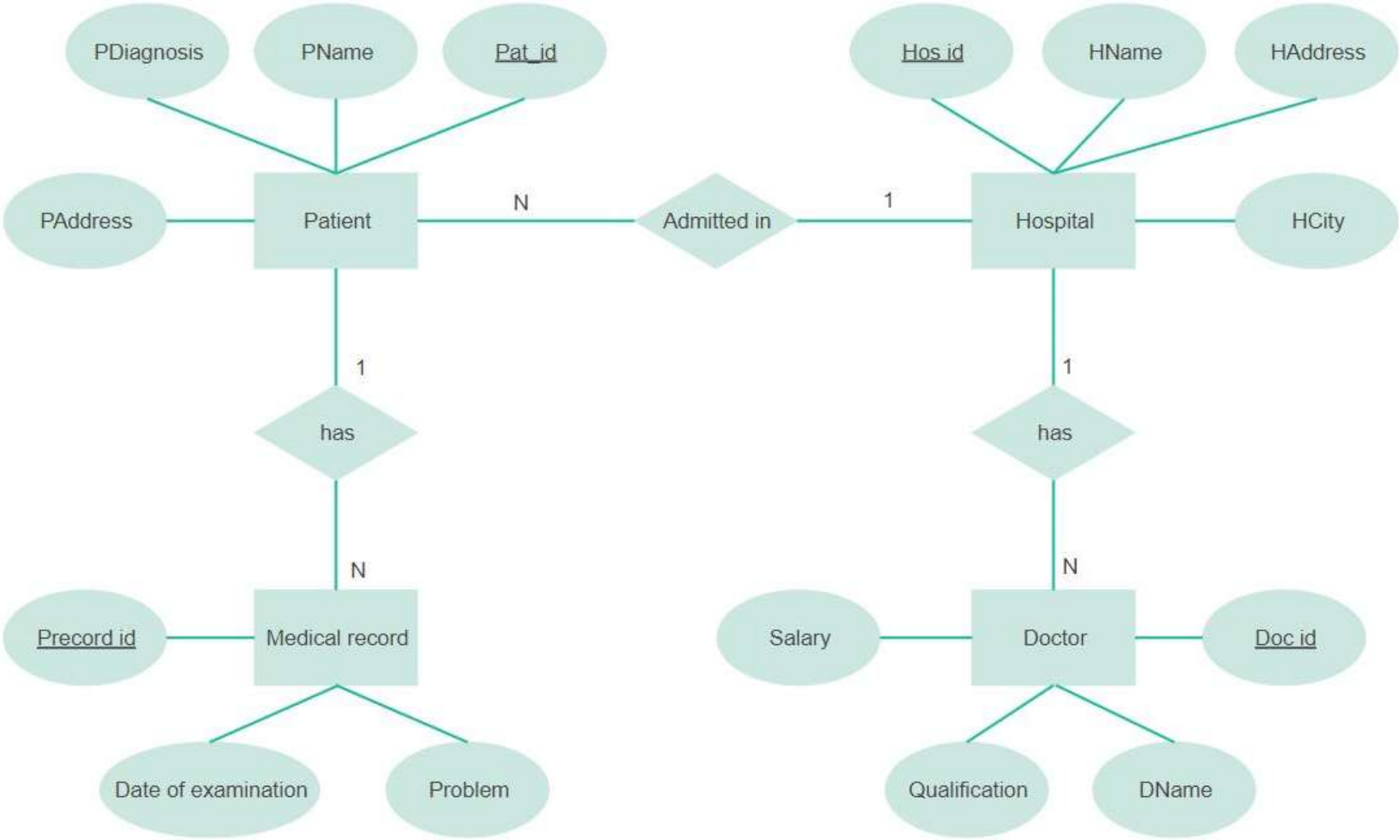
- While converting the relationship set to a table, the primary attributes of the two entity sets becomes the table attributes and if the relationship set has any attribute that also becomes the attribute of the table.
- For example :we have two entity sets Employee and Department. These entity sets are associated to each other using the Works relationship set. To convert this relationship set Works to the table, we take the primary attributes of each entity set, these are Emp_Id and Dept_Id and all the attributes of the relationship set and form a table.

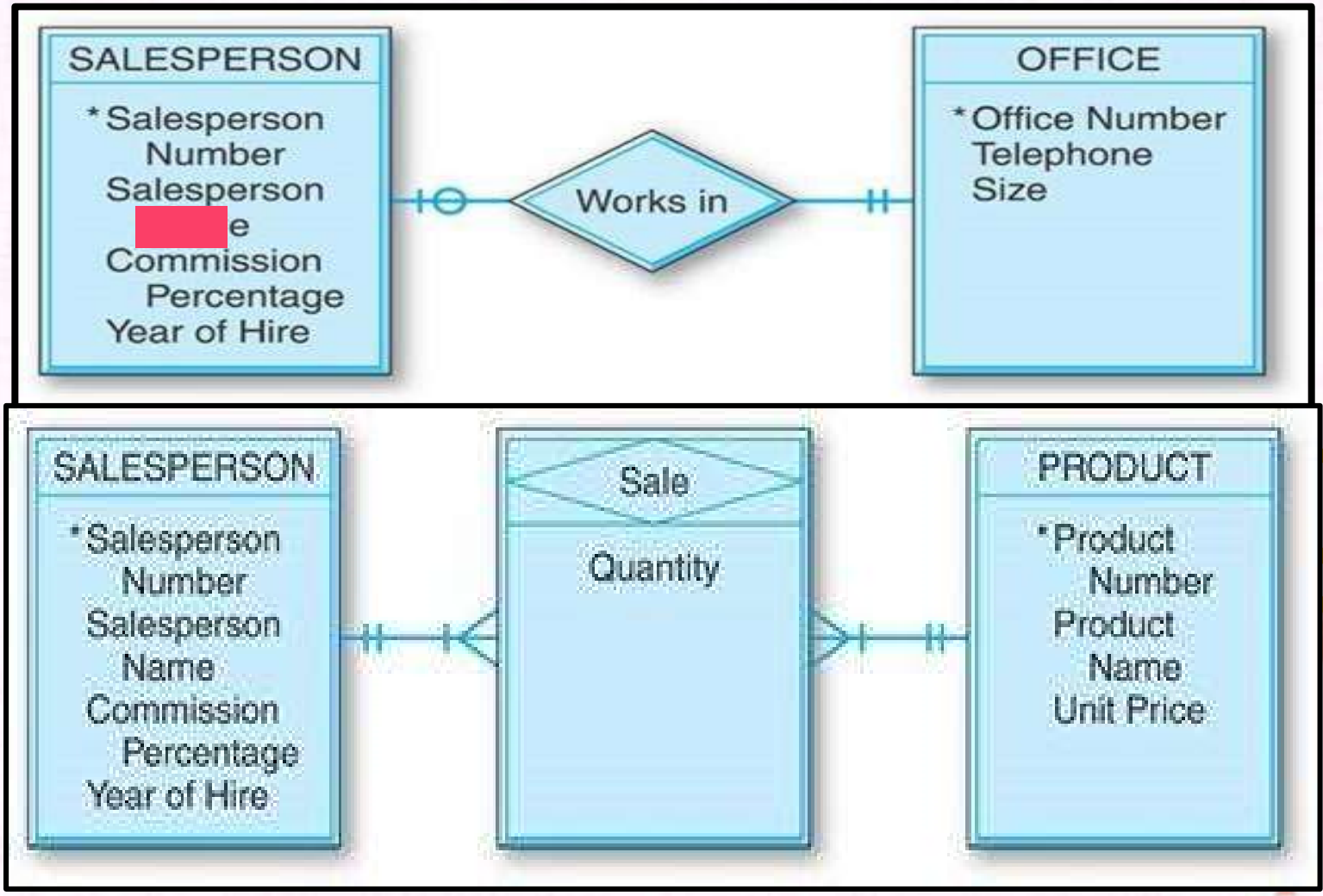

Name
Salary
Dept_Id
Dept_N
Employee
Works
Department
Emp_Id
Experience
No_Of_Emp
Emp_Id
Dept_Id
Experience
Beginnersbook.com



ER DIAGRAM OF A COMPANY

ER Diagram of Hospital Management System




SALESPERSON
*Salesperson Number
Salesperson [redacted]e
Commission Percentage
Year of Hire
Works in
OFFICE
*Office Number
Telephone
Size
SALESPERSON
*Salesperson Number
Salesperson Name
Commission Percentage
Year of Hire
Sale
Quantity
PRODUCT
*Product Number
Product Name
Unit Price

SQL Characteristics and advantages

- SQL stands for Structured Query Language
- SQL lets you access and manipulate databases
- SQL is a database language designed for the retrieval and management of data in a relational database.
- SQL is the standard language for database management
- All the RDBMS systems like MySQL, MS Access, Oracle, Sybase, Postgres, and SQL Server use SQL as their standard database language.
- SQL programming language uses various commands for different operations like DCL, TCL, DQL, DDL and DML.

SQL Data types
SQL Server String Data Type

char(n)	It is a fixed width character string data type. Its size can be up to 8000 characters.
varchar(n)	It is a variable width character string data type. Its size can be up to 8000 characters.
varchar(max)	It is a variable width character string data types. Its size can be up to 1,073,741,824 characters.
text	It is a variable width character string data type. Its size can be up to 2GB of text data.
nchar	It is a fixed width Unicode string data type. Its size can be up to 4000 characters.
nvarchar	It is a variable width Unicode string data type. Its size can be up to 4000 characters.

MySQL Date and Time Data Types

DATE	It is used to specify date format YYYY-MM-DD. Its supported range is from '1000-01-01' to '9999-12-31'.
DATETIME(fsp)	It is used to specify date and time combination. Its format is YYYY-MM-DD hh:mm:ss. Its supported range is from '1000-01-01 00:00:00' to 9999-12-31 23:59:59'.
TIMESTAMP(fsp)	It is used to specify the timestamp. Its value is stored as the number of seconds since the Unix epoch('1970-01-01 00:00:00' UTC). Its format is YYYY-MM-DD hh:mm:ss. Its supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC.
TIME(fsp)	It is used to specify the time format. Its format is hh:mm:ss. Its supported range is from '-838:59:59' to '838:59:59'
YEAR	It is used to specify a year in four-digit format. Values allowed in four digit format from 1901 to 2155, and 0000.

MySQL Numeric Data Types

BIT(Size)	It is used for a bit-value type. The number of bits per value is specified in size. Its size can be 1 to 64. The default value is 1.
INT(size)	It is used for the integer value. Its signed range varies from -2147483648 to 2147483647 and unsigned range varies from 0 to 4294967295. The size parameter specifies the max display width that is 255.
INTEGER(size)	It is equal to INT(size).
FLOAT(size, d)	It is used to specify a floating point number. Its size parameter specifies the total number of digits. The number of digits after the decimal point is specified by **d** parameter.
FLOAT(p)	It is used to specify a floating point number. MySQL used p parameter to determine whether to use FLOAT or DOUBLE. If p is between 0 to24, the data type becomes FLOAT (). If p is from 25 to 53, the data type becomes DOUBLE().
DOUBLE(size, d)	It is a normal size floating point number. Its size parameter specifies the total number of digits. The number of digits after the decimal is specified by d parameter.